Lecture 4
Decision Trees and Bagging

Machine Learning I

Alexander Quispe

ENEI – INEI · PEU-CD 2026

September 18, 2026

Outline

Decision Trees

Impurity and Information Gain

Growing and Pruning

Bagging

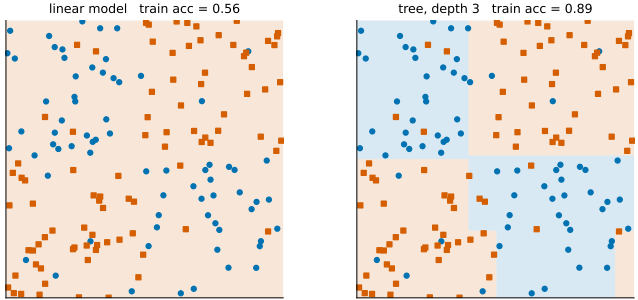
Summary

A Different Hypothesis Class

Three lectures on $\mathcal{H}_{\text{lin}}$. Its limitation is structural: a linear model cannot represent

$$y = \mathbf{1}\{x_1 > a \text{ and } x_2 < b\}\,, \qquad y = x_1 \cdot x_2, \qquad y = \text{XOR}(x_1, x_2)$$

without someone hand-crafting the right features. Trees learn the features.



Same data. Left: the best line. Right: a depth-3 tree. The tree's boundary is a union of axis-aligned boxes.

Decision Trees

The Function Class

A binary decision tree partitions $\mathbb{R}^p$ into M disjoint rectangles $R_1, \ldots, R_M$ by asking one question per internal node: *is* $x_j \leq s$*?* Each leaf m carries a constant prediction c_m:

$$f(\mathbf{x}) = \sum_{m=1}^{M} c_m \, \mathbf{1}\{\mathbf{x} \in R_m\} \, .$$

- **Regression:** $c_m \in \mathbb{R}$. **Classification:** $c_m \in \{1, \ldots, K\}$, or a vector of class proportions.
- Piecewise constant. Discontinuous at every split. No parameters in the usual sense — the "parameters" are the questions and their order.
- Invariant to monotone transformations of any single feature: $\log x_j$ and x_j produce the same tree. No standardization needed — a first for this course.

What Goes in a Leaf

Given the partition, choose the c_m to minimize empirical risk. The regions are disjoint, so this separates.

Regression, squared loss

$$\hat{c}_m = \arg\min_c \sum_{i:\,\mathbf{x}_i \in R_m} (y_i - c)^2, \qquad \frac{d}{dc}\sum_i (y_i - c)^2 = -2\sum_i (y_i - c) = 0$$

$$\Rightarrow \quad \hat{c}_m = \frac{1}{N_m} \sum_{i:\,\mathbf{x}_i \in R_m} y_i = \bar{y}_m.$$

Classification, 0/1 loss

With $\hat{p}_{mk} = \frac{1}{N_m}\sum_{i \in R_m} \mathbf{1}\{y_i = k\}$ the class proportions in the leaf,

$$\hat{c}_m = \arg\min_k \sum_{i \in R_m} \mathbf{1}\{y_i \neq k\} = \arg\min_k N_m(1 - \hat{p}_{mk}) = \arg\max_k \hat{p}_{mk}.$$

Leaves predict the mean (regression) or the majority (classification) of what fell into them. The whole difficulty is in choosing the partition.

How Many Trees Are There?

Even with the leaf values settled, the search space is enormous:

- A single split on feature j with N distinct values has $N-1$ candidate thresholds. With p features: $p(N-1)$ candidate first questions.
- A tree of depth d makes up to $2^d - 1$ such choices, and the choices interact.
- Finding the smallest tree consistent with a dataset is NP-complete (Hyafil & Rivest, 1976).

No algorithm finds the best tree. Every tree learner is **greedy**: pick the best single split now, recurse, never look back. What "best split" means is the content of the next section.

Impurity and Information Gain

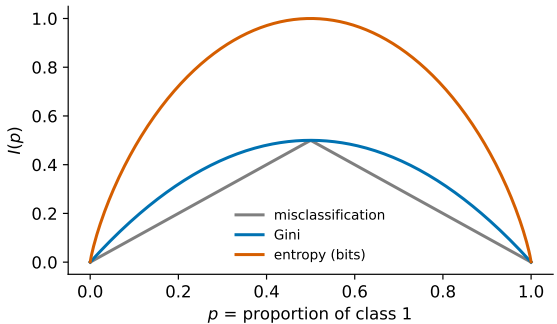
Measuring How Mixed a Node Is

For a node with class proportions $\mathbf{p} = (p_1, \ldots, p_K)$, an **impurity** $I(\mathbf{p}) \geq 0$ is 0 iff the node is pure ($p_k = 1$ for some k). Three standard choices:

Name	$I(\mathbf{p})$	Binary case ($p = p_1$)
Misclassification	$1 - \max_k p_k$	$\min(p, 1-p)$
Gini	$\sum_k p_k(1-p_k) = 1 - \sum_k p_k^2$	$2p(1-p)$
Entropy	$-\sum_k p_k \log_2 p_k$	$-p\log_2 p - (1-p)\log_2(1-p)$

- Misclassification is the 0/1 training error of the node's majority vote — the loss we care about.
- Gini is the probability that two draws from the node disagree; also the variance of a Bernoulli label, up to a factor 2. For regression, the analogue is the variance of y in the node.
- Entropy is the expected number of bits to encode a label drawn from the node.

The Three Curves



All three are symmetric about $p = \frac{1}{2}$, zero at the ends, maximal in the middle. Gini and entropy are strictly concave; misclassification is concave but piecewise *linear*. That difference matters.

Concavity

Gini and entropy are strictly concave on the simplex

Binary case: check the second derivative

$$I_G(p) = 2p - 2p^2, \qquad I''_G(p) = -4 < 0.$$

$$I_H(p) = -p\log_2 p - (1-p)\log_2(1-p), \qquad I''_H(p) = -\frac{1}{\ln 2}\left(\frac{1}{p} + \frac{1}{1-p}\right) < 0.$$

General K

Gini: $I_G(\mathbf{p}) = 1 - \mathbf{p}^\top \mathbf{p}$ has Hessian $-2\mathbf{I} \prec 0$. Entropy: $\partial^2 I_H / \partial p_k \partial p_l = -\delta_{kl}/(p_k \ln 2)$, a negative diagonal matrix.

Misclassification $\min(p, 1-p)$ is concave too, but with $I'' = 0$ except at $p = \frac{1}{2}$. It cannot tell apart two splits that move the same number of points across the majority line. Next slide.

Information Gain

Split node S (proportions $\mathbf{p}$, N points) into children S_L, S_R (proportions $\mathbf{p}_L, \mathbf{p}_R$; $N_L + N_R = N$).

Gain of a split

$$\Delta = I(\mathbf{p}) - \left[\frac{N_L}{N} I(\mathbf{p}_L) + \frac{N_R}{N} I(\mathbf{p}_R)\right]$$

$\Delta \geq 0$ for any concave I, with equality iff $\mathbf{p}_L = \mathbf{p}_R$ when I is strictly concave

Proof

The parent's proportions are the weighted average of the children's: $\mathbf{p} = \frac{N_L}{N}\mathbf{p}_L + \frac{N_R}{N}\mathbf{p}_R$ (count the points of each class). Jensen's inequality for concave I with weights $w_L = N_L/N$, $w_R = N_R/N$:

$$I\big(w_L \mathbf{p}_L + w_R \mathbf{p}_R\big) \;\geq\; w_L I(\mathbf{p}_L) + w_R I(\mathbf{p}_R),$$

which is exactly $\Delta \geq 0$. Strict concavity gives strict inequality unless $\mathbf{p}_L = \mathbf{p}_R$. □

A split never increases impurity. It fails to decrease it only when both children look like the parent.

Why Not Split on Misclassification Error?

Parent: 400 positives, 400 negatives. Two candidate splits:

	left child	right child	misclass. after	Gini after
Split A	$(300, 100)$	$(100, 300)$	$\frac{100+100}{800} = 0.250$	$\frac{400}{800} \cdot 0.375 + \frac{400}{800} \cdot 0.375 = 0.375$
Split B	$(200, 400)$	$(200, 0)$	$\frac{200+0}{800} = 0.250$	$\frac{600}{800} \cdot 0.444 + \frac{200}{800} \cdot 0 = 0.333$

- Misclassification: both splits leave 200 errors. Tie. It does not care that B produced a *pure* leaf.
- Gini: B scores $0.333 < 0.375$. B wins, because a pure child is worth something even if the other child got worse.
- Entropy agrees with Gini (0.811 vs 0.689 bits).

Splits are chosen for what they enable *later*. A pure leaf is done; a $(300, 100)$ leaf still needs work. Strictly concave impurities see this; the piecewise-linear one does not. Gini and entropy are used for growing; misclassification is used for pruning and reporting.

Numerical Example: One Split, Three Criteria

Parent S: 8 positives, 4 negatives. Split into $S_L = (6, 1)$ and $S_R = (2, 3)$.

Entropy

$$I_H(S) = -\tfrac{8}{12}\log_2\tfrac{8}{12} - \tfrac{4}{12}\log_2\tfrac{4}{12} = 0.9183$$

$$I_H(S_L) = -\tfrac{6}{7}\log_2\tfrac{6}{7} - \tfrac{1}{7}\log_2\tfrac{1}{7} = 0.5917, \qquad I_H(S_R) = -\tfrac{2}{5}\log_2\tfrac{2}{5} - \tfrac{3}{5}\log_2\tfrac{3}{5} = 0.9710$$

$$\Delta_H = 0.9183 - \left[\tfrac{7}{12}(0.5917) + \tfrac{5}{12}(0.9710)\right] = 0.9183 - 0.7498 = 0.1685 \text{ bits}$$

Gini

$$I_G(S) = 1 - \tfrac{64+16}{144} = 0.4444, \quad I_G(S_L) = 1 - \tfrac{36+1}{49} = 0.2449, \quad I_G(S_R) = 1 - \tfrac{4+9}{25} = 0.4800$$

$$\Delta_G = 0.4444 - \left[\tfrac{7}{12}(0.2449) + \tfrac{5}{12}(0.4800)\right] = 0.4444 - 0.3429 = 0.1016.$$

Misclassification

$$I_M(S) = \tfrac{4}{12}, \quad I_M(S_L) = \tfrac{1}{7}, \quad I_M(S_R) = \tfrac{2}{5}, \quad \Delta_M = \tfrac{4}{12} - \left[\tfrac{7}{12}\cdot\tfrac{1}{7} + \tfrac{5}{12}\cdot\tfrac{2}{5}\right] = \tfrac{4}{12} - \tfrac{3}{12} = 0.0833.$$

Growing and Pruning

Top-Down Greedy Training (CART)

Function `Grow(`S`)`**:**

  if *stopping rule holds for* S **then**

    return leaf with $\hat{c} = \bar{y}_S$ or $\arg\max_k \hat{p}_k$

  end

  foreach *feature* j *and threshold* s *among the midpoints of sorted* x_{ij} **do**

    $S_L \leftarrow \{i \in S : x_{ij} \le s\}, \quad S_R \leftarrow S \setminus S_L;$

    $\Delta(j,s) \leftarrow I(S) - \left[\frac{|S_L|}{|S|} I(S_L) + \frac{|S_R|}{|S|} I(S_R)\right];$

  end

  $(j^\star, s^\star) \leftarrow \arg\max \Delta(j,s);$

  return node asking "$x_{j^\star} \le s^\star$?" with children `Grow(`S_L`)`, `Grow(`S_R`)`;

- Cost per node: sort each feature once, $O(pN \log N)$, then a linear scan per feature updates the class counts incrementally. A full tree of depth d: $O(d\,pN \log N)$.
- Categorical features with q levels: $2^{q-1} - 1$ possible subsets. For binary y, sort levels by $\hat{p}$ and only $q - 1$ splits need checking (Breiman et al., 1984). Regression: same, sort by $\bar{y}$.
- Missing values: send them down a *surrogate* split, or treat "missing" as a category.

When to Stop

A tree grown until every leaf is pure has zero training error and memorizes the noise. Stopping rules are the tree's regularizers:

- **Maximum depth** $d_{\max}$ — at most $2^{d_{\max}}$ leaves.
- **Minimum samples per leaf** $n_{\min}$ — each leaf's $\hat{c}$ is an average of at least $n_{\min}$ labels, so its variance is at most $\sigma^2 / n_{\min}$.
- **Minimum gain** $\Delta_{\min}$ — do not split for less than this. Dangerous: a useless split can enable a very good one below it (XOR is the standard example).

Cost-complexity pruning (Breiman)

Grow a large tree T_0. For $\alpha \geq 0$, find the subtree $T \subseteq T_0$ minimizing

$$C_\alpha(T) = \sum_{m=1}^{|T|} N_m I(R_m) + \alpha |T|,$$

training impurity plus a charge per leaf. Larger α: smaller tree. The minimizing subtrees are nested as α grows, so the whole path is computed by repeatedly collapsing the internal node with the smallest increase in impurity per leaf removed (*weakest link*). Choose α by cross-validation.

Regression Trees

Same algorithm with the variance of y as impurity:

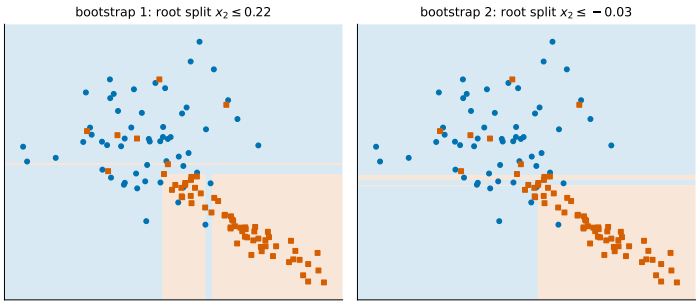
$$I(S) = \frac{1}{|S|}\sum_{i\in S}(y_i - \bar{y}_S)^2, \qquad \Delta(j,s) = I(S) - \left[\frac{|S_L|}{|S|}I(S_L) + \frac{|S_R|}{|S|}I(S_R)\right].$$

Maximizing Δ is minimizing the RSS after the split

$|S|\, I(S)$ is the RSS of predicting $\bar{y}_S$ on S. So $|S|\Delta = \mathrm{RSS}(S) - \left[\mathrm{RSS}(S_L) + \mathrm{RSS}(S_R)\right]$, and $\mathrm{RSS}(S)$ does not depend on the split. Maximizing Δ over (j,s) is minimizing $\mathrm{RSS}(S_L) + \mathrm{RSS}(S_R)$: the best split is the one whose two constant fits explain the most.

- Predictions are means of training responses, so a regression tree **cannot extrapolate**: outside the range of the training y, it is silent.
- The fitted function is a staircase. Smooth targets need many leaves; bagging (next) smooths the staircase.

Trees Are High-Variance



Two trees fitted to two bootstrap resamples of the same data. The top split changed; everything below it changed with it. In the language of Lecture 1: a deep tree has **low bias** (it can represent almost any function of the inputs) and **high variance** (which function it picks depends heavily on the sample).

Bagging

Averaging Reduces Variance

Lecture 1's decomposition says variance is the price of low bias. Averaging is the tool for variance.

B predictors, each with variance σ^2, pairwise correlation ρ

$$\mathrm{Var}\Big(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(\mathbf{x})\Big) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2.$$

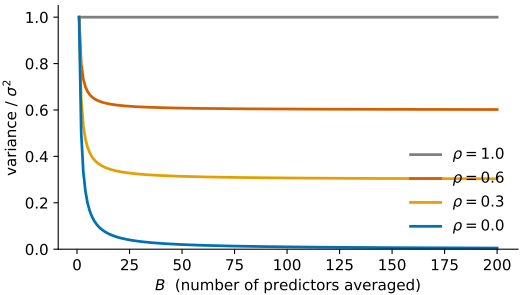
Proof

$$\mathrm{Var}\Big(\frac{1}{B}\sum_{b}\hat{f}_b\Big) = \frac{1}{B^2}\Big[\sum_{b}\mathrm{Var}(\hat{f}_b) + \sum_{b\neq b'}\mathrm{Cov}(\hat{f}_b,\hat{f}_{b'})\Big] = \frac{1}{B^2}\Big[B\sigma^2 + B(B-1)\rho\sigma^2\Big]$$

$$= \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2 = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \qquad \square$$

As $B \to \infty$ the second term vanishes and the first does not. Averaging can drive variance down to $\rho\sigma^2$, no further. Everything depends on making the predictors *different* from one another.

The Variance Floor



$\rho = 1$: identical predictors, averaging does nothing. $\rho = 0$: independent, variance $\to 0$ like $1/B$. Real bagged trees sit in between — they are fitted to overlapping data — so the curve flattens at $\rho\sigma^2$.

Bootstrap Aggregating (Breiman, 1996)

We have one sample, not B. Manufacture B by resampling:

---

for $b = 1, \ldots, B$ **do**

  draw $\mathcal{D}^{(b)}$: N points from $\mathcal{D}$ *with replacement*;

  fit a deep tree $\hat{f}_b$ to $\mathcal{D}^{(b)}$ (no pruning);

end

Output: $\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_b \hat{f}_b(\mathbf{x})$ (regression) or majority vote / averaged proportions (classification)

- Each $\hat{f}_b$ has the same distribution, so $\mathbb{E}[\hat{f}_{\text{bag}}] = \mathbb{E}[\hat{f}_1]$: **bias is unchanged**. Use deep trees — low bias — and let the average handle the variance.
- The bootstrap samples overlap, so $\rho > 0$. Typical values for trees: $\rho \in [0.3, 0.6]$.
- More trees never hurt: variance is non-increasing in B. $B = 100$–500 is usual; the curve flattens.

Out-of-Bag Error: Cross-Validation for Free

How much of the data does each tree see?

The probability that a given point is *not* drawn in one of N draws with replacement is

$$\left(1 - \frac{1}{N}\right)^N \xrightarrow[N\to\infty]{} e^{-1} \approx 0.368.$$

(Take logs: $N\log(1 - 1/N) = N(-1/N - 1/(2N^2) - \dots) \to -1$.)

So each tree is fitted on about 63% of the distinct points, and about 37% are **out of bag** for it.

- For each $\mathbf{x}_i$, average only the trees that did not see it: $\hat{f}_{\text{oob}}(\mathbf{x}_i)$. Roughly $B/3$ trees.
- The error of $\hat{f}_{\text{oob}}$ on the training set is an honest estimate of test error — every prediction is made by trees that never saw the point. No folds, no refitting.
- OOB error tracks the CV error of a bagged ensemble closely and costs nothing extra.

Random Forests (Breiman, 2001)

Bagging's floor is $\rho\sigma^2$. Lower ρ by making the trees see different *features*, not just different rows:

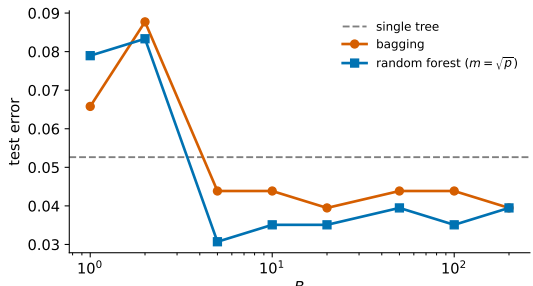
The one change

At every split, choose the best split among a random subset of m of the p features (fresh subset each node). Defaults: $m = \lfloor \sqrt{p} \rfloor$ for classification, $m = \lfloor p/3 \rfloor$ for regression.

- A dominant feature no longer appears at the top of every tree. Trees become less alike: ρ drops.
- Each tree gets slightly worse (higher σ^2, some bias); the ensemble gets better. The trade is governed by $\rho\sigma^2$, and ρ falls faster than σ^2 rises.
- $m = p$ recovers bagging. $m = 1$ chooses the feature at random and only optimizes the threshold.

Random forests do not overfit as B grows: the variance of the average is monotone in B and bounded below by $\rho\sigma^2$. They *can* overfit through the depth of the individual trees.

Bagging and Forests in Practice



Test error against B on a benchmark. A single tree (dashed) is the $B = 1$ value. Bagging drops quickly then flattens; the forest flattens lower. Neither curve turns back up.

Which Features Matter?

Trees make the question answerable:

- **Mean decrease in impurity.** Sum, over every node that splits on feature j, the gain Δ weighted by the node size; average over trees. Fast, but biased toward features with many distinct values.
- **Permutation importance.** Shuffle column j in the OOB data and measure how much OOB error rises. Model-agnostic and unbiased in the above sense; costs one pass per feature.

Importance measures association with the prediction, not causation, and they split credit arbitrarily among correlated features. Two copies of the same column each get half the importance.

Summary

What We Proved Today

1. Leaf values: the mean (regression) or the majority (classification) minimize the empirical risk on the leaf.
2. Gini and entropy are strictly concave; misclassification is only piecewise linear.
3. Gain $\Delta \geq 0$ for any concave impurity, by Jensen; strict when children differ. A strictly concave impurity rewards pure children even when the error count does not move.
4. Regression trees maximize the reduction in RSS. Leaves cannot extrapolate.
5. $\mathrm{Var}(\text{average of } B) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$: bagging removes the second term and leaves the first; random forests attack the first by lowering ρ.
6. Each bootstrap sample omits $\approx e^{-1}$ of the data, which gives an honest out-of-bag error.

Next

Lecture 5 (Monday) — the other way to combine trees: boosting. Instead of averaging many independent low-bias models, add many dependent high-bias models one at a time. Functional gradient descent, AdaBoost and its training-error bound, and why it keeps improving after the training error hits zero.

Lab 3 (Wednesday, Rodrigo) — information gain by hand, `min_samples_leaf` and overfitting, bagging vs. random forest vs. boosting on one dataset, and tuning all three.

Reading — *ESL* §9.2 (trees), §8.7 (bagging), §15.1–15.3 (random forests). Breiman (2001), *Random Forests*, Machine Learning 45.